

File Handling in C++

Writing, Reading, Appending, and Deleting Records

1. Introduction

Hello students! Today we are going to learn one of the most important and practical topics in C++ programming – **File Handling**.

Why this topic is needed?

When we write a simple C++ program, all the data we enter is stored in computer memory (RAM). As soon as the program finishes, the data is lost. But in real life, we need to keep records permanently. For example, a bank must keep customer account information even after the computer is switched off. A school must keep student records for years. This is where files come in.

Where is it used?

File handling is used everywhere – banking software, hospital management systems, library systems, online shopping websites, mobile apps, and even your contact list on your phone.

Why do programmers use it?

Programmers use files to store data permanently. It allows reading back the saved data, updating it, or deleting it when needed. Without file handling, no real-world application can be built.

Why do companies use it?

Companies need to store huge amounts of information – customer details, transaction logs, employee records, invoices. Files (and databases built on files) are the backbone of

business data storage. Even cloud storage ultimately uses files. So learning file handling is essential for any software developer.

Let us start from the very basics and move forward step by step. You will be able to write programs that create files, read from them, add new data, and even delete specific records – all with proper understanding.

2. Learning Objectives

At the end of this chapter, students will be able to:

- Understand what files are and why they are needed.
 - Write data into a file using C++.
 - Read data from an existing file and display it on the screen.
 - Append new data at the end of a file without erasing old data.
 - Delete a particular record from a file using the temporary file technique.
 - Differentiate between various file opening modes (`ios::out`, `ios::in`, `ios::app`).
 - Solve exam problems and viva questions confidently.
 - Design small file-based projects.
-

3. Very Easy Definition

General File Handling:

File handling means saving information permanently on the computer so that it can be used again even after the program stops.

Writing Data into Files:

Putting information from the program into a file so that it gets saved on the disk.

Reading Data from Files:

Taking information from a saved file and bringing it into the program to display or process.

Appending Data into Files:

Adding new information at the end of an already existing file without removing old data.

Deleting Records from Files:

Removing a specific piece of information (one line or one record) from the file safely.

4. Real Life Analogy

Think of a **School Attendance Register**.

- At the beginning of the year, the teacher **writes** the names of all students (Writing data).
- Every day, the teacher **reads** the register to see who is present (Reading data).
- When a new student joins the class later, the teacher **adds** the name at the bottom of the list without erasing the old names (Appending data).
- If a student leaves the school, the teacher **removes** (cuts or strikes off) that student's name (Deleting a record).

This is exactly what we do with computer files.

5. Why Do We Need This?

Before using files (using only variables and arrays):

Without Files

Problem

Data is stored in variables

Lost when program ends

Data stored in arrays

Size is fixed, lost after execution

Can't keep records permanently

No history

Every time you run the program, you must enter data again

Time waste, error prone

After using files:

With Files	Benefit
Data is saved on hard disk	Permanent storage
Data remains after program ends	Can be accessed later
File size can be large	Handle thousands of records
Enter data once, use many times	Saves time, reliable
Supports append and delete operations	Full record management
So, file handling turns a temporary program into a real-world application.	

6. Syntax

Let us learn the syntax for the four main operations one by one. You must understand every word.

6.1 Writing Data (Creating a new file)

```
ofstream file("students.txt");  
file << "Ali" << endl;  
file.close();
```

Explanation of each part:

- `ofstream` – output file stream class. It creates an object that can write data to a file.
- `file` – the object name; you can give any name (e.g., `fout`, `outFile`).
- `"students.txt"` – the name of the file to be created on disk. The double quotes mean it is a string `()`.
- `( and )` – constructor arguments that open the file.
- `;` – end of statement.
- `<<` – insertion operator, same as `cout` but here it sends data into the file.

- "Ali" – the actual data we want to store.
- endl – inserts a new line in the file.
- file.close() – closes the file and saves it properly.

6.2 Reading Data

```
ifstream file("students.txt");
string name;
getline(file, name);
cout << name;
file.close();
```

Explanation:

- ifstream – input file stream class. Used to read data from a file.
- file – object name for reading.
- getline(file, name) – reads one complete line from the file and stores it into the string variable name.
- cout << name – displays the data on screen.
- file.close() – closes the file after reading.

6.3 Appending Data

```
ofstream file("students.txt", ios::app);
file << "Ahmed" << endl;
file.close();
```

Explanation:

- ios::app – "append mode". It tells the computer: "Do not erase old data; just add new data at the end."
- Rest same as writing.

6.4 Deleting Records

Deletion is not a direct command. We use a **temporary file method**.

Concept steps (algorithm):

1. Open the original file in read mode.
2. Open a temporary file in write mode.
3. Read records one by one.
4. If the record is **not** the one to delete, write it into the temporary file.
5. If it is the record to delete, skip it (do not write).
6. Close both files.
7. Delete the original file.
8. Rename the temporary file to the original file's name.

We will see the full code later. The syntax uses `remove()` and `rename()` functions from `<cstdio>`.

7. Required Header Files

For file handling, we must include these libraries at the top of our program:

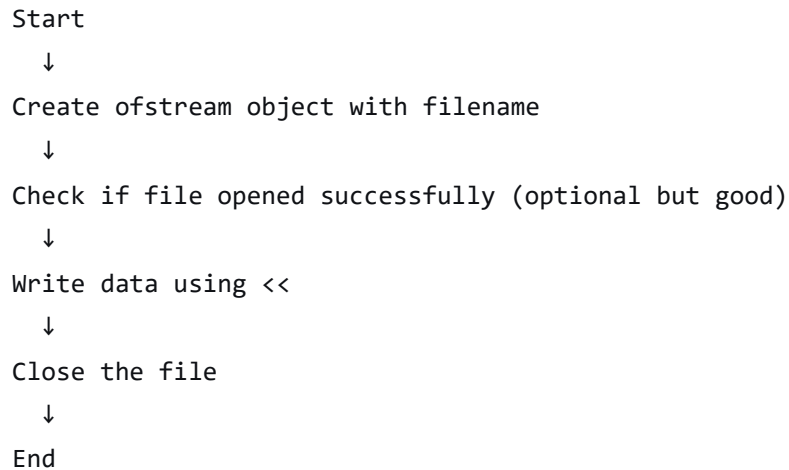
```
#include <iostream>    // for cout, cin, endl
#include <fstream>      // for ofstream, ifstream, fstream
#include <string>        // for string data type
#include <cstdio>        // for remove() and rename() (needed in deletion)
using namespace std;   // to avoid writing std:: every time
```

Purpose of each:

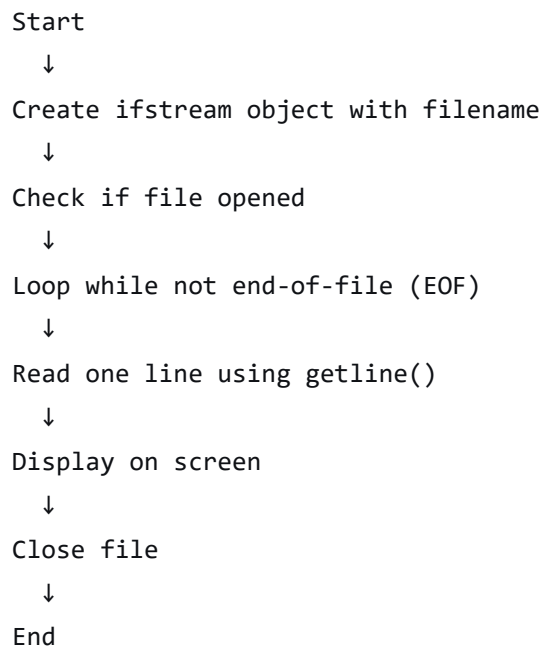
- `<iostream>` – gives us console input/output (`cout`, `cin`).
 - `<fstream>` – gives us file stream classes: `ofstream` (write), `ifstream` (read), `fstream` (both read/write).
 - `<string>` – allows us to use the `string` data type easily.
 - `<cstdio>` – provides functions to delete a file (`remove`) and rename a file (`rename`). These are old C-style functions but still very useful.
 - `using namespace std;` – makes code shorter by not writing `std::cout`, `std::ifstream`, etc.
-

8. Flow Diagram

Flowchart for Writing Data



Flowchart for Reading Data



Flowchart for Appending Data

Start



Create ofstream object with ios::app



Write new data (it goes to end automatically)



Close file



End

Flowchart for Deleting a Record

Start



```
Open original file for reading
```



```
Open temp file for writing
```



Read each record from original



Is this the record to delete? – Yes → Skip (don't write)



No → Write to temp file



Close both files



```
remove("original.txt");
```

```
rename("temp.txt", "original.txt");
```



End

9. Step-by-Step Working (Line by Line Explanation)

I will explain a very basic write program in detail.

Program: Write “Hello” to a file

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream myFile("greeting.txt"); // Line 1
    myFile << "Hello, Students!" << endl; // Line 2
    myFile.close(); // Line 3
    cout << "Data written successfully." << endl;
    return 0;
}
```

Line-by-line explanation:

- `#include <iostream>` and `#include <fstream>`: Including libraries we already discussed.
- `using namespace std;`: So we can use `cout`, `ofstream` without `std::`.
- `int main()`: Program starts here.
- **Line 1:** `ofstream myFile("greeting.txt");`
 - We create an output file stream object named `myFile`.
 - Inside the brackets, we pass the filename `"greeting.txt"`. If this file does not exist, it will be created automatically. If it exists already, its old content will be **erased** (because this is default write mode `ios::out`).
 - Now `myFile` is connected to `greeting.txt` on the disk.
- **Line 2:** `myFile << "Hello, Students!" << endl;`
 - The `"Hello, Students!"` goes into the file. `endl` puts a newline so that next writing starts on the next line.
- **Line 3:** `myFile.close();`
 - Closes the file. This is important to save data properly. If you do not close, sometimes the data may not be written completely.
- `cout << ...` – Prints a message on the screen so the user knows the operation is done.

Dry Run (Mental execution):

Before program runs, `greeting.txt` does not exist or is empty.

After Line 1, file is opened. Memory: `myFile` object holds connection.

After Line 2, `"Hello, Students!\n"` is stored in the file's buffer.

After Line 3, buffer is flushed and file saved.

Output on screen: "Data written successfully."

If we open `greeting.txt` with Notepad, we see "Hello, Students!".

10. Basic Example (The Easiest Program)

10.1 Writing a Single Word

Program:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream out("oneword.txt");
    out << "Pakistan";
    out.close();
    return 0;
}
```

Output: No screen output. Check file: contains `Pakistan`.

Explanation:

The word "Pakistan" is written into `oneword.txt`. Very straightforward.

10.2 Reading a Single Word (using >>)

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream in("oneword.txt");
```

```

string word;
in >> word;    // reads one word (stops at space or newline)
cout << "Read: " << word << endl;
in.close();
return 0;
}

```

Output: Read: Pakistan

Dry Run:

File contains Pakistan. in >> word extracts the word into variable word. Then cout displays.

Important: >> reads only one word. It stops at space or newline.

11. Second Example – Store Student Name, Roll Number, Marks

Writing multiple fields:

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream f("student.txt");
    f << "Ali" << endl;
    f << 101 << endl;
    f << 85.5 << endl;
    f.close();
    cout << "Student record saved." << endl;
    return 0;
}

```

Reading the same:

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream f("student.txt");
    string name;
    int roll;
    float marks;
    getline(f, name);    // reads "Ali"
    f >> roll;           // reads 101
    f >> marks;          // reads 85.5
    cout << "Name: " << name << endl;
    cout << "Roll: " << roll << endl;
    cout << "Marks: " << marks << endl;
    f.close();
    return 0;
}

```

Output:

```

Name: Ali
Roll: 101
Marks: 85.5

```

Why getline for name and >> for numbers?

Because >> does not read spaces; if name had space (e.g., "Ali Ahmad"), >> would only read "Ali". `getline` reads the whole line including spaces.

12. Third Example – Employee Record (Real-World)

Let's create a small program to store employee name, ID, and salary into a file.

```

#include <iostream>

```

```

#include <fstream>
#include <string>
using namespace std;

int main() {
    ofstream fout("employees.txt");
    string name;
    int id;
    double salary;

    cout << "Enter employee name: ";
    getline(cin, name);
    cout << "Enter ID: ";
    cin >> id;
    cout << "Enter salary: ";
    cin >> salary;

    fout << name << endl;
    fout << id << endl;
    fout << salary << endl;

    fout.close();
    cout << "Employee record saved successfully." << endl;
    return 0;
}

```

Sample Run:

```

Enter employee name: Sana Khan
Enter ID: 1122
Enter salary: 45000
Employee record saved successfully.

```

File employees.txt contains:

```

Sana Khan
1122
45000

```

This is a real-world scenario used in companies.

13. Reading Data (Detailed Explanation)

We already saw basic reading. Now let's understand reading from a file containing multiple records.

13.1 Reading Line by Line Until End of File (EOF)

Assume file `cities.txt` has:

Lahore
Karachi
Islamabad

Program:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream fin("cities.txt");
    string city;
    while (getline(fin, city)) { // reads line, loop continues until EOF
        cout << city << endl;
    }
    fin.close();
    return 0;
}
```

Output:

Lahore
Karachi
Islamabad

How does the loop work?

`getline(fin, city)` returns the stream object `fin`. When the stream is in a boolean context (like `while`), it returns `true` if the read was successful, and `false` when EOF (end of file) is reached. So the loop automatically stops when there are no more lines.

Difference between `>>` and `getline()`:

`>>` operator

Reads until space/newline

Leaves the newline character in the stream

Good for numbers and single words

`getline()`

Reads entire line including spaces

Consumes the newline

Good for strings with spaces

Common Mistake: Mixing `>>` and `getline()`. After reading a number with `>>`, the newline remains in the input buffer, so the next `getline()` will read that empty newline. Solution: add `cin.ignore()` or `fin.ignore()` before `getline`.

14. Appending Data (ios::app)

Appending means **adding new data at the end** without deleting old data.

14.1 Program to Append to an Existing File

Suppose `notes.txt` already contains:

Meeting at 10 AM

We run:

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream fout("notes.txt", ios::app);
    fout << "Meeting at 2 PM" << endl;
    fout.close();
    cout << "New line appended." << endl;
    return 0;
}

```

Now `notes.txt` becomes:

Meeting at 10 AM
Meeting at 2 PM

14.2 Difference Between Overwrite and Append

Action	Mode	Old data	Result
Overwrite	<code>ofstream f("file.txt");</code> or <code>ios::out</code>	Erased	File has only new data
Append	<code>ofstream f("file.txt", ios::app);</code>	Kept	New data added at end

Remember: If you open a file with only `ofstream f("file.txt");` and write something, the old content is destroyed. So if you want to keep old data, always use `ios::app`.

14.3 Appending Student Record Example

```
ofstream f("students.txt", ios::app);
```



```
f << "Fatima" << endl;
f << 102 << endl;
f << 90.0 << endl;
f.close();
```

Now `students.txt` will have a new student after existing ones.

15. Deleting Records

Direct deletion of a line from a file is **not possible** in standard C++ without loading the whole file into memory. The simple and safe method used in exams and university curriculum is the **Temporary File Method**.

15.1 Why Direct Deletion is Impossible

Files are sequential streams. You cannot remove a line from the middle; you can only read or write sequentially. Therefore, we read all lines, skip the unwanted, and write the rest to a new file, then replace the original.

15.2 Temporary File Method – Algorithm

1. Open original file for reading (`ifstream`).
2. Open a temporary file for writing (`ofstream`).
3. While there are records in original file:
 - Read one record.
 - If this record is **NOT** the one to delete, write it to temp file.
 - If it **IS** the record to delete, skip writing it.
4. Close both files.
5. Delete original file using `remove("original.txt")`.
6. Rename temp file to original file name using `rename("temp.txt", "original.txt")`.

15.3 Example: Delete a Specific Student Name

Assume `names.txt` has:

Ali
Ahmed
Sara

We want to delete "Ahmed".

Full Code:

```
#include <iostream>
#include <fstream>
#include <string>
#include <cstdio> // for remove and rename
using namespace std;

int main() {
    ifstream fin("names.txt");
    ofstream fout("temp.txt");
    string line;
    string target = "Ahmed";

    while (getline(fin, line)) {
        if (line != target) {
            fout << line << endl;
        }
    }

    fin.close();
    fout.close();

    remove("names.txt");
    rename("temp.txt", "names.txt");

    cout << "Record deleted successfully (if it existed)." << endl;
    return 0;
}
```

Dry Run:

- Original file: Ali, Ahmed, Sara.
- Iteration 1: line = "Ali" → not target → write to temp.
- Iteration 2: line = "Ahmed" → is target → skip.
- Iteration 3: line = "Sara" → write.
- After loop: temp contains Ali, Sara.
- Remove original, rename temp to original. Now `names.txt` has Ali and Sara.

15.4 Deletion of a Student Record by Roll Number (Advanced)

Suppose a file `students.txt` stores records in three lines per student: name, roll, marks.
To delete a student with roll 102.

Program:

```
#include <iostream>
#include <fstream>
#include <string>
#include <cstdio>
using namespace std;

int main() {
    ifstream fin("students.txt");
    ofstream fout("temp.txt");
    string name, rollStr, marksStr;
    int targetRoll = 102;

    while (getline(fin, name)) {
        getline(fin, rollStr);
        getline(fin, marksStr);
        int currentRoll = stoi(rollStr); // convert string to int
        if (currentRoll != targetRoll) {
            fout << name << endl;
            fout << rollStr << endl;
            fout << marksStr << endl;
        }
    }
    fin.close();
    fout.close();
}
```

```

remove("students.txt");
rename("temp.txt", "students.txt");
cout << "Record with roll " << targetRoll << " deleted if found." << endl;
return 0;
}

```

Explanation of new things:

- `stoi(rollStr)` converts string to integer. Need `#include <string>` (already there). If using older compilers, you may need `atoi(rollStr.c_str())`, but `stoi` is modern.
 - We read three lines because each record has three fields.
 - Only write those records whose roll number does not match.
-

16. Dry Run (Detailed for Deletion Example)

Let's dry run the roll-number deletion code with sample data:

Initial `students.txt`:

```

Ali
101
85.5
Ahmed
102
90.0
Sara
103
78.0

```

Target roll = 102

Execution steps:

- `fin` opens `students.txt`, `fout` opens `temp.txt` (empty).
- **First student:**
 - `getline(fin, name) → name = "Ali"`
 - `getline(fin, rollStr) → rollStr = "101"`

- `getline(fin, marksStr) → marksStr = "85.5"`
- `currentRoll = stoi("101") = 101`
- `101 != 102 → write all three lines to temp.`
- **Second student:**
- `name = "Ahmed", rollStr = "102", marksStr = "90.0", currentRoll=102.`
- `102 == 102 → skip` (do not write).
- **Third student:**
- `name = "Sara", roll = "103", marks = "78.0", currentRoll=103.`
- `103 != 102 → write to temp.`
- close both.
- `remove("students.txt") → original deleted.`
- `rename("temp.txt", "students.txt") → temp becomes new students.txt.`

Final `students.txt`:

```
Ali
101
85.5
Sara
103
78.0
```

Record of Ahmed (roll 102) deleted.

17. Output Explanation

For the deletion program, the screen says: `Record with roll 102 deleted if found.` If the roll number existed, it got removed; if not, the file remains unchanged. The message is informational. The actual effect is seen when you open the file.

18. Internal Working (What Happens Inside Computer Memory)

- **Opening a file for writing (ofstream):** Operating system creates a buffer in RAM. Your data goes into that buffer. When buffer is full or file closed, data is written to disk (hard drive).
- **Reading (ifstream):** A buffer is allocated; disk data is read in chunks. `getline` extracts until newline. File pointer moves automatically.
- **Appending (ios::app):** File pointer is positioned at end of file before any write. So all writes go after existing data.
- **Deletion:** We don't really delete inside the original file. We copy everything except the unwanted record to a new file, then swap names. The original file's space is freed by `remove()`.

File pointer: Every open file has a hidden pointer indicating where the next read or write will occur. In write mode it starts at beginning (0). In append mode it starts at end. Each read/write moves it ahead.
